面向高速端到端视觉避障的 ViT+Mamba 时序建模与流式部署一致性分析

戴英特

2026 年 6 月

中图分类号： TQ028.1

UDC分类号： 540

面向高速端到端视觉避障的 ViT+Mamba 时序建模与流式部署一致性分析

作 者 姓 名 戴英特

学 院 名 称 自动化学院

指 导 教 师 甘明刚教授

答辩委员会主席 _____

申 请 学 位 工学硕士

学 科 专 业 控制工程

学位授予单位 北京理工大学

论文答辩日期 2026 年 6 月

ViT+Mamba Temporal Modeling and Streaming Deployment Consistency Analysis for High-Speed End-to-End Visual Obstacle Avoidance

Candidate Name:	Dai Yingte
School or Department:	School of Automation
Faculty Mentor:	Prof. Gan Minggang
Chair, Thesis Committee:	
Degree Applied:	Master
Major:	Control Engineering
Degree by:	Beijing Institute of Technology
The Date of Defence:	June, 2026

面向高速端到端视觉避障的ViT+Mamba时序建模与流式部署一致性分析

戴英特

北京理工大学

研究成果声明

本人郑重声明：所提交的学位论文是我本人在指导教师的指导下进行的研究工作获得的研究成果。尽我所知，文中除特别标注和致谢的地方外，学位论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得北京理工大学或其它教育机构的学位或证书所使用过的材料。与我一同工作的合作者对此研究工作所做的任何贡献均已在学位论文中作了明确的说明并表示了谢意。

特此申明。

作者签名：_____　　签字日期：_____

关于学位论文使用权的说明

本人完全了解北京理工大学有关保管、使用学位论文的规定，其中包括：①学校有权保管、并向有关部门送交学位论文的原件与复印件；②学校可以采用影印、缩印或其它复制手段复制并保存学位论文；③学校可允许学位论文被查阅或借阅；④学校可以学术交流为目的，复制赠送和交换学位论文；⑤学校可以公布学位论文的全部或部分内容（保密学位论文在解密后遵守此规定）。

作者签名：_____　　导师签名：_____

签字日期：_____　　签字日期：_____

主要符号对照表

D_t	第 t 个控制周期的深度图像观测，$D_t \in \mathbb{R}^{H\times W}$
s_t	第 t 个控制周期的轻量状态向量，$s_t = [q_t, \tilde{v}^{\text{target}}]$
o_t	第 t 个控制周期的完整观测，$o_t = (D_t, s_t)$
q_t	无人机在世界坐标系下的姿态四元数，$q_t = [w, x, y, z]$
$\tilde{v}^{\text{target}}$	归一化目标前向速度，$\tilde{v}^{\text{target}} = v^{\text{target}}/10$
$\mathbf{v}_t$	第 t 个控制周期的速度指令，$\mathbf{v}_t = [v_t^x, v_t^y, v_t^z] \in \mathbb{R}^3$
$\mathbf{v}_{\text{raw}}$	策略网络原始输出速度指令
$\mathbf{v}_{\text{cmd}}$	经 RACS 约束后最终发布的速度指令
$\mathbf{v}_{\text{prev}}$	上一控制步发布的速度指令
π_θ	参数为 θ 的端到端策略网络
π^*	特权信息专家策略
$\mathbf{h}_t$	序列模型（Mamba/LSTM）在第 t 步的内部隐状态
$\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}$	连续时间状态空间模型的系统矩阵
$\bar{\mathbf{A}}, \bar{\mathbf{B}}$	零阶保持离散化后的状态空间参数矩阵
Δ	Mamba 选择性机制中的输入相关离散化步长
δ_t	RACS 动态速率上界
$d_{\min,t}$	第 t 步深度图像中的最小深度观测值
$\mathcal{L}_{\text{BC}}$	行为克隆监督损失（MSE）
$\mathcal{L}_{\text{jerk}}$	指令抖动惩罚损失
λ_{jerk}	Jerk Loss 权重系数
β	DAgger 中的专家混合比例
T	回合总帧数 / 序列长度
$\tau_{\max}$	最大回合时长（40 s）
d_{model}	Mamba 模块的模型维度（192）
d_{state}	Mamba 模块的状态维度（64）
BC	行为克隆（Behavioral Cloning）
DAgger	数据集聚合（Dataset Aggregation）
RACS	动态速率限制控制平滑器（Rate-Adaptive Control Smoother）

SSM	结构化状态空间模型（Structured State Space Model）
ViT	视觉 Transformer（Vision Transformer）
ID	同分布（In-Distribution）
OOD	分布外（Out-of-Distribution）

目 录

摘要 I
Abstract III
主要符号对照表 V
第 1 章 绪论 1
1.1 研究背景与应用场景 1
1.1.1 高速闭环的延迟预算分析 2
1.1.2 高速下的误差放大机制 2
1.1.3 端到端范式需要时序建模的直觉 4
1.2 问题定义 6
1.2.1 输入-输出接口 6
1.2.2 部分可观测性与序列建模的必要性 7
1.3 关键挑战 8
1.3.1 挑战一：时序建模与高速决策 8
1.3.2 挑战二：流式部署一致性 8
1.3.3 挑战三：算力约束与高效视觉骨干 9
1.4 研究贡献 10
1.5 论文结构安排 11
第 2 章 预备知识与相关工作 13
2.1 四旋翼控制接口与任务抽象 13
2.1.1 坐标系与控制量定义 13
2.1.2 任务形式化 14
2.1.3 控制回路与低层控制器假设 14
2.1.4 安全指标与任务完成条件 15

2.2 模仿学习与分布偏移：BC 与 DAgger 15
2.2.1 行为克隆（BC） 15
2.2.2 分布偏移与误差累积 15
2.2.3 DAgger：数据集聚合 16
2.3 视觉表征：CNN 与 ViT 18
2.3.1 卷积神经网络 18
2.3.2 视觉 Transformer（ViT） 18
2.3.3 轻量化 ViT 的设计维度 19
2.4 时序建模：RNN/LSTM 与 SSM 20
2.4.1 LSTM 的流式优势与局限 20
2.4.2 结构化状态空间模型（S4） 20
2.4.3 从连续到离散的零阶保持（ZOH）推导 21
2.4.4 Mamba 的选择性机制 21
2.4.5 SSM 对控制任务的意义 23
2.5 MambaVision：混合 Mamba-Transformer 视觉骨干 24
2.6 仿真平台与数据来源 25
2.6.1 Flightmare 仿真平台 25
2.6.2 评测环境 25
2.6.3 特权信息专家策略 25
2.6.4 数据采集管线 27
2.7 评测协议与指标 27
2.7.1 统一评测协议 27
2.7.2 指标定义 27
2.7.3 指标计算伪代码 28
2.7.4 统计显著性与不确定性报告 28
2.7.5 评测可审计规范 29

2.8 相关工作综述 30
2.8.1 端到端视觉飞行控制 30
2.8.2 模块化自主飞行 31
2.8.3 安全性与部署侧约束 31
2.8.4 Sim-to-Real 迁移 32
2.8.5 方法谱系总结 32
2.9 小结：设计需求 32
第 3 章 面向高速避障的 ViT+Mamba 端到端策略 34
3.1 第 3 章引言 34
3.2 问题形式化 35
3.3 模型结构 35
3.3.1 总体架构 35
3.3.2 视觉编码器：轻量化 ViT 36
3.3.3 时序聚合模块：Temporal Mamba 36
3.3.4 特征融合与控制头 37
3.3.5 基线设置：ViT+LSTM 37
3.3.6 设计选择讨论 37
3.3.7 复杂度与推理延迟分析 38
3.4 DAgger 闭环数据增强 38
3.4.1 动机与方法 38
3.4.2 执行配置 39
3.4.3 数据集构成与速度档分布 39
3.4.4 轮次收敛的机制解释 39
3.5 RACS 动态速率限制平滑器 40
3.5.1 背景与动机 40
3.5.2 约束建模与求解 40
3.5.3 δ_t 动态调度函数 41
3.5.4 边界情况与失败模式分析 42

3.5.5 与训练期 Jerk Loss 的关系 42
3.5.6 算法复杂度与 RACS 伪代码 43
3.6 训练细节 43
3.7 案例研究与实验 44
3.7.1 主结果：ViT+Mamba 与 ViT+LSTM 的系统对比 45
3.7.2 推理延迟与系统时序分析 46
3.7.3 DAgger 轮次实验：碰撞频次与方差收敛 47
3.7.4 RACS：安全–平滑权衡 49
3.7.5 消融实验 51
3.7.6 失败案例分析 52
3.8 第 3 章小结 53
第 4 章 流式部署一致性与状态生命周期管理 55
4.1 第 4 章引言 55
4.2 Batch–Streaming 等价性定义 56
4.2.1 Batch 训练模式 56
4.2.2 Streaming 推理模式 57
4.2.3 等价性条件 57
4.2.4 SSM 线性递推的等价性推导 58
4.2.5 对 LSTM/GRU 与 Transformer KV-Cache 的类比 58
4.3 状态生命周期协议 60
4.3.1 错误状态重置的退化机理 60
4.3.2 回合边界级状态管理协议 60
4.3.3 实现细节 61
4.3.4 常见工程错误与症状对照 62
4.4 等价性单测与硬防护机制 63
4.4.1 等价性单测 63
4.4.2 阈值选择依据 63
4.4.3 硬防护机制 64

4.5 案例研究与实验 64
4.5.1 KeepState 与 ResetState 对比 65
4.5.2 LSTM 的状态重置退化 66
4.5.3 漂移可视化 66
4.5.4 等价性单测结果 68
4.5.5 重置频率消融 68
4.5.6 部署 burn-in 消融 69
4.5.7 跨速度泛化验证 70
4.6 第 4 章小结 71
第 5 章 全 SSM 架构探索：MambaVision 替代的系统消融实验 73
5.1 实验目标与评价问题定义 73
5.2 实验环境、数据与评测协议 75
5.2.1 评测环境与速度档 75
5.2.2 指标体系 75
5.2.3 系统架构与控制变量 75
5.3 消融实验框架与对照组 76
5.3.1 对照组与实验组定义 76
5.3.2 消融验证原则 76
5.3.3 控制变量声明 77
5.4 深度归一化一致性验证 77
5.4.1 训练与评测归一化口径差异 77
5.4.2 输入分布偏移机制分析 78
5.4.3 定量验证结果 78
5.4.4 实验规范锁定 80
5.5 输入分辨率瓶颈验证 81
5.5.1 分辨率瓶颈分析 81
5.5.2 实验设置 82
5.5.3 定量结果 82

5.5.4 速度段差异分析 83
5.5.5 计算开销评估 84
5.5.6 小结 84
5.6 视觉骨干受控解冻实验 85
5.6.1 动机与控制变量声明 85
5.6.2 实验配置 85
5.6.3 结果分析 86
5.6.4 原因讨论 87
5.6.5 结论与局限性 88
5.7 结构改动与动作蒸馏实验 88
5.7.1 特征条件池化 89
5.7.2 动作知识蒸馏 89
5.7.3 高速段尾部风险分析 91
5.7.4 保守归因与局限性 92
5.8 风险状态补分布蒸馏 93
5.8.1 核心假设 93
5.8.2 方法：风险状态采集与加权微调 93
5.8.3 高速段蒸馏效果验证 95
5.9 低速补强与最终模型综合评估 97
5.9.1 “平滑性提升却导致碰撞率增加”的失败模式 97
5.9.2 教师 Omega 事件触发的低速补强 99
5.9.3 版本对照与控制变量 100
5.9.4 Spheres 环境综合对比 101
5.9.5 Trees（OOD）泛化验证 104
5.9.6 推理效率与部署可行性 104
5.9.7 综合数据表 105
5.9.8 迭代路径总结 107
5.10 第 5 章小结 107

结论 109
参考文献 112
攻读学位期间发表论文与研究成果清单 118
致谢 119

插 图

图 1.1 四旋翼无人机高速避障的典型应用场景与共性挑战 1
图 1.2 高速闭环控制中的误差放大链路 3
图 1.3 高速飞行中的误差放大与累积示意：深度噪声与运动模糊导致实际轨迹偏离理想轨迹，误差随时间步持续放大 3
图 1.4 模块化导航范式与端到端控制范式的对比：左侧为传统模块化流水线（Sensor→SLAM→Mapping→Planning→Low-Level Control），延迟逐级累积；右侧为端到端学习控制，传感器输出经神经网络直接映射为控制指令，显著降低延迟 5
图 1.5 端到端视觉避障任务闭环控制流程 7
图 1.6 训练 Batch 模式与部署 Streaming 模式的状态管理差异，以及错误的逐步重置导致的无记忆退化 9
图 1.7 自注意力机制 $O(n^2)$ 与 SSM 线性 $O(n)$ 复杂度的对比示意 10
图 1.8 该研究技术路线总览 11
图 2.1 世界坐标系与机体坐标系定义，以及速度指令接口 13
图 2.2 行为克隆（BC）端到端训练流程：左侧由专家策略 π^* 在环境中采集观测–动作对构成数据集 $\mathcal{D}$；中间将序列观测输入端到端神经网络 π_θ 预测动作 $\hat{a}_t$；右侧通过 MSE 损失 $\mathcal{L} = \|a_t - \hat{a}_t\|^2$ 计算梯度并反向传播更新网络参数 16
图 2.3 DAgger 数据聚合闭环流程 17
图 2.4 DAgger 迭代式数据聚合全流程示意：上层为环境交互阶段，混合策略 $\hat{\pi}_i = \beta_i \pi^* + (1-\beta_i)\pi_i$ 在环境中采集轨迹并由专家 π^* 修正标注；中层为数据聚合阶段，新采集数据 $\mathcal{D}_{\text{new}}$ 与历史数据集 $\mathcal{D}_i$ 合并；下层为训练更新阶段，以聚合数据集重训策略 π_{i+1}。右侧对比图展示 BC 误差 $O(T^2)$ 增长与 DAgger 误差 $O(1)$ 收敛的理论差异 17
图 2.5 ViT 的 patch token 化与 Transformer 编码流程示意 19

图 2.6 SSM/Mamba 的三层架构总览。上层：连续时间状态空间方程 $\mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}\mathbf{x}(t)$，其中 $\mathbf{A}$ 驱动状态演化，$\mathbf{B}$ 控制输入注入，$\mathbf{C}$ 负责状态读出；中层：基于零阶保持（ZOH）的离散化过程，将连续参数转化为离散递推形式 $\bar{\mathbf{A}} = e^{\mathbf{A}\Delta}$；下层：Mamba 的选择性机制，展示了参数 Δ_t、$\mathbf{B}_t$、$\mathbf{C}_t$ 如何依赖输入 $\mathbf{x}_t$ 进行动态调制。右侧示意图类比了其自适应控制逻辑与 LSTM 门控机制的异同。 22

图 2.7 SSM/Mamba 离散化后的状态更新机制。下标 t 强调了参数 $\bar{\mathbf{B}}_t$ 与 $\mathbf{C}_t$ 随输入动态变化的选择性特性。 23

图 2.8 Attention、SSM 与 LSTM 的序列长度–计算量关系对比（示意） 24

图 2.9 Flightmare 仿真平台中两类评测环境的实拍截图。(a) Spheres 环境：三维空间中随机分布不同半径的球体障碍，训练数据在该环境中生成；(b) Trees 环境：由树干与冠层构成的自然场景，策略从未在此环境中训练，用于测试零样本迁移泛化能力 26

图 2.10 基于 Flightmare 与特权信息专家的数据采集管线 27

图 2.11 碰撞事件次数的上升沿检测计算示意 29

图 3.1 第 3 章组织结构：架构设计、训练增强与部署约束三线汇聚于实验验证 34

图 3.2 ViT+Mamba 端到端策略网络总体架构 35

图 3.3 RACS 几何投影示意：将超出约束的 $\mathbf{v}_{\text{raw}}$ 投影至 δ_t-球面边界 41

图 3.4 $\delta_t(d_{\min,t})$ 调度曲线示意 42

图 3.5 ViT+Mamba 与 ViT+LSTM 在 Spheres/Trees 环境下的性能对比。（a）全程碰撞率；（b）碰撞事件次数；（c）Command Jerk；（d）单步推理时间。 45

图 3.6 控制循环周期 Δt 的分布统计，验证不同方法的系统时序一致性 47

图 3.7 碰撞事件次数随 DAgger 轮次变化（9 m/s 与 12 m/s），误差条表示 10 次试验标准差 47

图 3.8 平均碰撞持续时间随 DAgger 轮次变化 48

图 3.9 碰撞指标跨试验标准差随 DAgger 轮次的演化 48

图 3.10 BC 基线与 DAgger-R3 在 ID/OOD 环境下碰撞率对比 49

图 3.11 Trees OOD 环境下 BC 与 DAgger-R3 的多维指标对比 49

图 3.12 碰撞事件次数的逐轮分布 49

图 3.13 碰撞持续时间的逐轮分布（条件分布，duration > 0） 49

图 3.14 RACS 验证：Mamba No RACS、Mamba + RACS 与 ViT+LSTM 在不同速度下的安全性与平滑性 50
图 3.15 不同方法的 Command Jerk 随速度变化趋势 51
图 4.1 第 4 章逻辑链路：从问题揭示到形式化、协议设计与实验验证 56
图 4.2 状态重置导致的级联退化因果链 60
图 4.3 状态生命周期状态机：Init→Warmup→Run→Terminate→Reset 61
图 4.4 Batch 模式与正确/错误 Streaming 模式的时间轴对比 61
图 4.5 KeepState 与 ResetState 的漂移对比。ResetState（红色）导致显著的横向漂移趋势，而 KeepState（绿色）的轨迹保持稳定。 66
图 4.6 KeepState 与 ResetState 模式下横向漂移的累积对比 67
图 4.7 Batch–Streaming 等价性测试：KeepState 下 $\Delta\mathbf{v}_t$ 在 10^{-6} 量级（蓝色），ResetState 下 $\Delta\mathbf{v}_t$ 在 10^{-1} 量级（红色），两者差 5 个数量级 68
图 4.8 重置频率消融实验结果：重置越频繁，碰撞率、Jerk 与横向漂移越高。 . 69
图 4.9 部署 burn-in 消融实验结果：适度 burn-in 对碰撞率改善有限，但可稳定降低 Jerk 与漂移。 70
图 4.10 KeepState 与 ResetState 跨速度碰撞率对比：ResetState 的碰撞率随速度上升快速放大。 70
图 4.11 KeepState 与 ResetState 跨速度横向漂移对比：ResetState 在全速度域均表现出更大的横向漂移。 71
图 5.1 逐步消融验证决策流程图。每一阶段验证单一假设，通过则锁定该变量进入下一阶段，失败则记录归因并探索替代路线。最终收敛到可用版本 74
图 5.2 MambaVision 替代实验系统架构。虚线框标注的视觉编码器为唯一变量，其余模块（时序 Mamba、控制头、部署协议）在所有实验中保持固定 76
图 5.3 深度归一化口径偏移对速度效率与碰撞事件的影响。(a) 各速度段的平均前向速度在 png255 模式下均显著低于 threshold 模式，12 m/s 段降幅达 34%；(b) 5 m/s 段碰撞事件次数从 72.2 降至 13.6 属速度退化导致的表面改善，12 m/s 段碰撞事件次数反而从 7.6 升至 15.0，揭示控制衰减而非鲁棒性提升的本质 79

图 5.4　输入分辨率从 60×90 提升至 128×192 后的碰撞事件与速度效率变化。(a) 碰撞事件次数在所有速度段均显著下降，5 m/s 段降幅达 72.9%；(b) 各速度段平均前向速度均有提升，推理延迟仅从约 20 ms 增至 21 ms（+5%） 83

图 5.5　Stage 3 解冻前后的安全–速度权衡散点图（mean collision count vs mean V_x）。红色点（Stage 3 解冻方案）相比蓝色点（冻结高分辨率方案）在所有速度段均向右移动（碰撞增加），9 m/s 与 12 m/s 段速度基本持平，表明碰撞恶化并非速度变化的副产物。灰色点为 ViT+Mamba 基线 86

图 5.6　各消融阶段在 5 m/s、9 m/s、12 m/s 三个速度段的碰撞事件次数 (a) 与平均前向速度 (b) 对比。星号标注该速度段最优方法。特征条件池化（FC-Pool）与动作蒸馏（ActionKD）均在 5 m/s 段碰撞略有改善，但在 9 m/s 与 12 m/s 段碰撞次数均高于冻结基线（蓝色），速度效率在各方法间无显著差异 90

图 5.7　12 m/s 高速段各方法逐试验碰撞次数分布。菱形标注均值，水平线标注中位数，虚线为冻结基线中位数（= 6）作为基准参照。动作蒸馏（103-B KD）的标准差达 7.4，且出现单次试验碰撞次数超过 25 的极端离群值 92

图 5.8　风险状态蒸馏各阶段碰撞率对比。V_mid（橙色）相比 V_frozen（蓝色）在 9 m/s 与 12 m/s 段碰撞率显著下降，但 3 m/s 与 5 m/s 段碰撞率大幅上升。V_final（绿色）在全速度段均达到最低碰撞率 95

图 5.9　风险状态蒸馏各阶段平均前向速度对比。三个版本的 Mean V_x 高度接近，确认碰撞率变化并非降速的副产物 96

图 5.10　5 m/s 场景下不同补强策略的碰撞率对比。与中间版本（V_mid）相比，基于 $d_{\min}$ 触发的补强版本（V_fail）碰撞率不降反升（从 4.66% 升至 5.67%，增幅 21.7%），而基于 Omega 事件触发的最终版本则显著降至 0.56%。 98

图 5.11　5 m/s 场景下不同补强策略的指令抖动对比。V_fail 版本的 Jerk 从 0.08 锐减至 0.01（降幅 87.5%），控制输出极度平滑但碰撞率却发生恶化，呈现出典型的“平滑失效”特征。 98

图 5.12　Spheres 环境碰撞率对比。MV+Mamba 最终版在所有速度段碰撞率均不高于基线，12 m/s 高速段碰撞率降幅达 55.5% 101

图 5.13　Spheres 环境碰撞事件次数对比。MV+Mamba 最终版在 5 m/s–12 m/s 段碰撞事件次数均低于基线 102

图 5.14　Spheres 环境单位距离碰撞密度对比。MV+Mamba 最终版在高速段 CPM 优于基线，7 m/s 段差异在误差范围内 102
图 5.15　Spheres 环境平均前向速度对比。MV+Mamba 最终版在所有速度段均高于基线，证明安全性优势并非通过降速实现 103
图 5.16　Spheres 环境指令抖动对比。MV+Mamba 最终版在多数速度段 Jerk 显著更低，且碰撞率同步改善，构成安全性与平滑性的双重提升 103
图 5.17　Trees（OOD）环境碰撞率对比。MV+Mamba 最终版在中高速段（7 m/s–12 m/s）碰撞率大幅优于基线，体现显著的 OOD 泛化优势 104
图 5.18　推理延迟对比。MV+Mamba 最终版推理延迟约 21 ms–23 ms，高于基线（约 14 ms–17 ms），但仍远低于 50 ms 实时控制阈值 105
图 5.19　从冻结高分辨率基线到 MV+Mamba 最终版的迭代链路与分支决策。左侧主线给出四阶段迭代，其中数据侧转向框内并入 (§5.8) 与 (§5.9) 的关键细节；右侧汇总 (§5.6–5.7) 中已验证但否决的替代路线及其失败结论。 107

表 格

表 1.1 端到端避障系统延迟预算分解 2
表 2.1 不同输入分辨率与 Patch Size 下的 Token 数量与注意力复杂度 19
表 2.2 SSM 特性与高速避障控制需求的映射 23
表 2.3 评测环境配置 25
表 2.4 特权信息专家策略超参数 26
表 2.5 统一评测协议 27
表 2.6 评测指标定义与计算口径 28
表 2.7 高速端到端视觉避障相关技术路线对比 32
表 3.1 ViT 编码器各阶段张量尺寸 36
表 3.2 Temporal Mamba 模块配置 36
表 3.3 各模块推理耗时分析（RTX 4060，float32，单帧） 38
表 3.4 DAgger 迭代执行配置 39
表 3.5 DAgger 各轮新增轨迹的速度档分布 39
表 3.6 RACS 超参数 42
表 3.7 策略网络训练超参数配置 44
表 3.8 Spheres 环境主结果：碰撞率与碰撞事件次数（均值 ±std） 46
表 3.9 Trees 环境主结果：碰撞率与碰撞事件次数（均值 ±std） 46
表 3.10 Mamba 层数消融（Spheres，9 m/s，10 次均值） 51
表 3.11 Burn-in 长度消融（Spheres，9 m/s，10 次均值） 52
表 3.12 Jerk Loss 配置消融（Spheres，9 m/s，10 次均值） 52
表 4.1 不同序列模型的内部状态与错误重置后果类比 59
表 4.2 常见状态管理工程错误与症状对照 62
表 4.3 等价性测试配置与通过标准 64
表 4.4 可审计日志字段定义 65
表 4.5 Mamba 流式状态管理消融实验（KeepState vs ResetState，Spheres 7 m/s） 65

表 4.6　LSTM 流式状态管理消融实验（KeepState vs ResetState，Spheres 7 m/s）　66
表 5.1　MambaVision 替换实验控制变量声明 77
表 5.2　深度归一化口径一致性验证定量结果（均值 ± 标准差，每档 10 次试验）。threshold 为训练阶段配置，png255 为口径偏移配置。加粗行标注最终锁定的归一化模式 79
表 5.3　MambaVision-T 各 Stage 空间 token 数量随输入分辨率的变化。低分辨率下深层 Stage 的 token 数量急剧退化，构成空间表征能力的结构性瓶颈 81
表 5.4　输入分辨率瓶颈验证定量结果（每档 10 次试验）。60×90 为基线低分辨率配置，128×192 为提升后配置。加粗行标注最终锁定的分辨率 82
表 5.5　视觉骨干受控解冻实验配置。相比前一阶段（冻结高分辨率方案），唯一变化为 Stage 3 由冻结变为可训练（加粗标注） 86
表 5.6　风险状态蒸馏中间版本（V_mid）评测结果。加粗标注各速度段最优值。V_mid 在高速段（9 m/s、12 m/s）碰撞率大幅改善，但低速段（3 m/s、5 m/s）碰撞率显著恶化 96
表 5.7　风险状态蒸馏各版本对照表。每行仅标注相对前一版本的唯一变化 100
表 5.8　推理效率与模型参数对比 105
表 5.9　Spheres 环境综合对比（均值 ±std，每档 10 次试验）。加粗标注各速度段各指标的优势方 106
表 5.10　Trees（OOD）环境综合对比（均值 ±std，每档 10 次试验） 106

第 4 章　流式部署一致性与状态生命周期管理

4.1　第 4 章引言

第 3 章的实验结果证实，ViT+Mamba 策略在高速工况下的表现显著优于 ViT+LSTM 基线。然而，这一优越性能的复现建立在一个关键的隐式前提之上：序列模型的内部状态（Internal States）在流式部署过程中必须得到正确的管理与维护。第 4 章将系统揭示一个常被忽视却极其危险的陷阱——当状态管理机制失效时，策略的碰撞率将从 0% 飙升至 90%，导致系统彻底失效。

端到端控制系统在实际部署时遵循流式（Streaming）计算范式：每个控制周期仅接收当前时刻的观测输入，并实时输出控制指令。这与训练阶段采用的定长序列批次并行计算（Batch Mode）存在本质差异。若在工程实现中错误地在每次推理调用时重置隐状态，具备时序记忆能力的序列模型将瞬间退化为 “无记忆策略”——等效于一个仅依赖当前帧输入的反应式控制器。

这一问题在现有的端到端控制文献中鲜有系统性探讨。大多数研究默认部署实现的正确性，但在实际工程落地中，状态管理错误往往具有极强的隐蔽性：即使状态被错误重置，策略仍可能输出数值合理且平滑的控制指令，在低速简单场景下甚至能维持 “伪正常” 的避障表现；唯有在高速或高动态复杂环境中，累积的记忆缺失才会导致灾难性的性能崩溃。

第 4 章的主要贡献在于：

1. 给出了 Batch–Streaming 计算等价性的严格定义与数学推导；
2. 剖析了常见状态管理错误的工程症状与诊断方法；
3. 提出了基于回合边界（Episode Boundary）的状态生命周期管理协议与硬防护机制；
4. 通过 KeepState（状态保持）与 ResetState（状态重置）的对比实验及多维消融分析，定量证实了该问题的毁灭性后果（详见第4.5节）。

图 4.1给出第 4 章的逻辑链路。

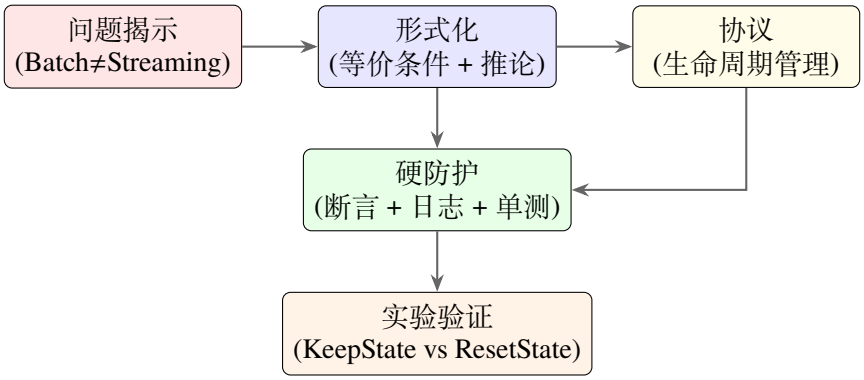

图 4.1　第 4 章逻辑链路：从问题揭示到形式化、协议设计与实验验证

4.2　Batch–Streaming 等价性定义

该节将形式化定义序列模型的两种核心计算范式：训练阶段的 Batch 模式与部署阶段的 Streaming 模式，并推导两者等价的充要条件。

4.2.1　Batch 训练模式

在训练阶段，策略网络以定长序列（例如 $T = 150$ 步）为单位进行前向计算。序列模型接收完整的输入序列 $\{\mathbf{x}_1, \ldots, \mathbf{x}_T\}$，利用并行扫描（如 Mamba）或循环展开（如 LSTM）机制，一次性并行计算出所有时间步的输出。该模式的关键特征在于：1. 全序列可见：模型可同时访问序列中的所有时刻数据；2. 状态生命周期：内部状态在序列起始处初始化，在序列内部连续传播，并在序列计算结束后被重置或释放。

以 Mamba 为例，Batch 模式下的计算过程可展开为如下形式：

$$
\begin{aligned}
\mathbf{h}_1^{\text{batch}} &= \bar{\mathbf{A}}\mathbf{h}_0 + \bar{\mathbf{B}}_1\mathbf{x}_1, \quad \mathbf{y}_1^{\text{batch}} = \mathbf{C}_1\mathbf{h}_1^{\text{batch}} \\
\mathbf{h}_2^{\text{batch}} &= \bar{\mathbf{A}}\mathbf{h}_1^{\text{batch}} + \bar{\mathbf{B}}_2\mathbf{x}_2, \quad \mathbf{y}_2^{\text{batch}} = \mathbf{C}_2\mathbf{h}_2^{\text{batch}} \\
&\vdots \\
\mathbf{h}_T^{\text{batch}} &= \bar{\mathbf{A}}\mathbf{h}_{T-1}^{\text{batch}} + \bar{\mathbf{B}}_T\mathbf{x}_T, \quad \mathbf{y}_T^{\text{batch}} = \mathbf{C}_T\mathbf{h}_T^{\text{batch}}
\end{aligned} \tag{4.1}
$$

其中 $\mathbf{h}_0$ 为初始状态（通常设为零向量）。在实际实现中，上述过程通常通过高效的并行扫描算法 [82] 完成。

4.2.2 Streaming 推理模式

在部署阶段，系统以流式（Streaming）方式运行。每个控制周期仅输入当前时刻的观测 $\mathbf{x}_t$，递推更新内部状态 $\mathbf{h}_t$ 并输出 $\mathbf{y}_t$。该模式的关键特征在于：1. 单帧处理：每一步仅处理当前时刻的数据；2. 状态持久化：内部状态必须跨控制周期持续传播与维护；3. 因果限制：模型无法访问未来的输入信息。

Streaming 模式下第 t 步的计算公式为：

$$\begin{aligned}\mathbf{h}_t^{\text{stream}} &= \bar{\mathbf{A}}\mathbf{h}_{t-1}^{\text{stream}} + \bar{\mathbf{B}}_t\mathbf{x}_t \\ \mathbf{y}_t^{\text{stream}} &= \mathbf{C}_t\mathbf{h}_t^{\text{stream}}\end{aligned} \tag{4.2}$$

其中 $\mathbf{h}_{t-1}^{\text{stream}}$ 是从上一个控制周期继承并保留下来的内部状态。

4.2.3 等价性条件

基于上述定义，给出 Batch–Streaming 等价性的严格形式化描述。

定义 4.1 (Batch–Streaming 等价性)**.** 对于给定的序列模型 f，若在相同的初始状态 $\mathbf{h}_0$ 和相同的输入序列 $\{\mathbf{x}_1, \ldots, \mathbf{x}_T\}$ 下，Batch 模式产生的输出序列 $\{\mathbf{y}_1^{\text{batch}}, \ldots, \mathbf{y}_T^{\text{batch}}\}$ 与 Streaming 模式产生的输出序列 $\{\mathbf{y}_1^{\text{stream}}, \ldots, \mathbf{y}_T^{\text{stream}}\}$ 满足：

$$\mathbf{y}_t^{\text{batch}} = \mathbf{y}_t^{\text{stream}}, \quad \forall t \in \{1, \ldots, T\} \tag{4.3}$$

则称该模型的两种计算模式是等价的。

当且仅当以下三个条件同时满足时，两种模式的输出严格等价：

C1 初始化一致性：内部状态 $\mathbf{h}_0$ 的初始化方式必须完全一致；

C2 传播连续性：在同一回合（Episode）内，状态的更新过程不得被中断或重置；

C3 输入一致性：输入序列的内容与顺序必须严格一致（这通常依赖于预处理流水线的确定性）。

4.2.4 SSM 线性递推的等价性推导

推论 4.1 (SSM Streaming 等价于 Batch 扫描的逐步展开)**.** 对于 Mamba 的线性递推关系 $\mathbf{h}_t = \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}_t\mathbf{x}_t$，在条件 C1–C3 成立的前提下，Streaming 模式在数学上等价于 Batch 并行扫描过程的逐步展开。

证明. 采用数学归纳法证明。

基础情形 ($t = 1$)：根据公式 (4.2)：

$$\mathbf{h}_1^{\text{stream}} = \bar{\mathbf{A}}\mathbf{h}_0 + \bar{\mathbf{B}}_1\mathbf{x}_1 \tag{4.4}$$

由条件 C1 知 $\mathbf{h}_0^{\text{stream}} = \mathbf{h}_0^{\text{batch}}$，由条件 C3 知 $\mathbf{x}_1$ 相同。对比公式 (4.1) 可知 $\mathbf{h}_1^{\text{stream}} = \mathbf{h}_1^{\text{batch}}$，故等式成立。

归纳步骤：假设 $\mathbf{h}_{t-1}^{\text{stream}} = \mathbf{h}_{t-1}^{\text{batch}}$ 成立。由条件 C2（状态未被重置）和 C3（$\mathbf{x}_t$ 相同），推导第 t 步状态：

$$\mathbf{h}_t^{\text{stream}} = \bar{\mathbf{A}}\underbrace{\mathbf{h}_{t-1}^{\text{stream}}}_{=\mathbf{h}_{t-1}^{\text{batch}}} + \bar{\mathbf{B}}_t\mathbf{x}_t = \bar{\mathbf{A}}\mathbf{h}_{t-1}^{\text{batch}} + \bar{\mathbf{B}}_t\mathbf{x}_t = \mathbf{h}_t^{\text{batch}} \tag{4.5}$$

由于 $\mathbf{h}_t^{\text{stream}} = \mathbf{h}_t^{\text{batch}}$，则直接可得输出 $\mathbf{y}_t^{\text{stream}} = \mathbf{C}_t\mathbf{h}_t^{\text{stream}} = \mathbf{C}_t\mathbf{h}_t^{\text{batch}} = \mathbf{y}_t^{\text{batch}}$。 □

该推论的逆否命题为系统故障诊断提供了理论依据：若观测到 $\mathbf{y}_t^{\text{stream}} \neq \mathbf{y}_t^{\text{batch}}$，则必然意味着条件 C1、C2 或 C3 中至少有一项被违反。这为定位状态管理缺陷提供了形式化的逻辑基础。

4.2.5 对 LSTM/GRU 与 Transformer KV-Cache 的类比

为了展示第 4 章理论的普适性，将上述结论推广至其他主流序列模型。

推论 4.2 (LSTM/GRU 的等价性条件)**.** LSTM 的递推方程涉及隐状态 $\mathbf{h}_t$ 和细胞状态 $\mathbf{c}_t$

的联合更新：

$$
\begin{aligned}
\mathbf{f}_t &= \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \\
\mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t)
\end{aligned} \tag{4.6}
$$

其中 $\mathbf{f}_t, \mathbf{i}_t, \mathbf{o}_t$ 分别为遗忘门、输入门和输出门。等价性条件 C1–C3 在此同样适用，但对于条件 C2，必须保证状态元组 $(\mathbf{h}_{t-1}, \mathbf{c}_{t-1})$ 作为一个整体进行联合传播——任一分量的丢失或重置都将破坏计算的等价性。在该研究的 ViT+LSTM 基线实验中（第4.5.2节），ResetState 同样导致了碰撞率的显著恶化，证实了 LSTM 对状态重置的敏感性与 Mamba 相当。

推论 4.3 (Transformer KV-Cache 的类比)**.** 自回归 Transformer 在增量推理（Incremental Inference）时维护 Key-Value 缓存（KV-Cache）。在每一步推理中，当前 Token 的 Key 和 Value 被追加到缓存中：

$$
\text{KV-Cache}_t = \text{Concat}(\text{KV-Cache}_{t-1}, [\mathbf{K}_t, \mathbf{V}_t]) \tag{4.7}
$$

若该缓存结构在推理过程中被错误清空，Transformer 将丧失对历史 Token 的注意力机制，退化为仅关注当前 Token 的模型。这在功能上与 Mamba/LSTM 的状态重置是完全等价的。

表 4.1 总结了三类主流序列模型在状态管理层面的类比关系。

表 4.1　不同序列模型的内部状态与错误重置后果类比

模型架构	**内部状态形式**	**传播方式**	**重置后退化结果**
SSM (Mamba)	隐状态 $\mathbf{h}_t \in \mathbb{R}^d$	线性递推	无记忆 MLP
LSTM/GRU	$(\mathbf{h}_t, \mathbf{c}_t) \in \mathbb{R}^{2d}$	门控递推	无记忆 MLP
Transformer	KV-Cache $\in \mathbb{R}^{L\times 2d}$	缓存拼接	单 Token 注意力

这一分析表明，第 4 章提出的状态生命周期管理协议具有跨架构的普适性：任何依赖跨步状态传递的序列模型，在流式部署中都面临着相同的工程风险。

4.3 状态生命周期协议

4.3.1 错误状态重置的退化机理

当内部状态在每个控制步被重置为零向量时，递推方程退化为：

$$\mathbf{h}_t^{\text{reset}} = \bar{\mathbf{A}} \cdot \mathbf{0} + \bar{\mathbf{B}}_t \mathbf{x}_t = \bar{\mathbf{B}}_t \mathbf{x}_t \tag{4.8}$$

模型输出仅取决于当前帧 $\mathbf{x}_t$，完全丧失历史记忆。这引发级联效应：

1. 时序聚合失效：模型无法利用前几帧的运动信息估计障碍的相对运动方向，避障决策仅基于当前深度快照；
2. 控制指令抖动加剧：相邻帧的深度观测存在传感器噪声，无记忆模型对噪声的逐帧放大导致输出抖动显著增加；
3. 系统性横向漂移：抖动指令的统计偏差（例如由相机安装偏差引起的系统性深度偏移）在无历史修正的情况下被持续放大，表现为宏观轨迹漂移；
4. 碰撞率急剧上升：上述三个效应叠加，在高速密集障碍环境中导致碰撞率从接近 0% 飙升至 90%。

图 4.2以因果链形式展示了这一退化过程。

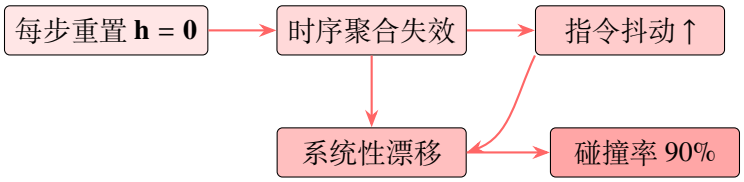

图 4.2　状态重置导致的级联退化因果链

4.3.2 回合边界级状态管理协议

协议的核心原则为：序列模型的内部状态仅在回合边界进行初始化，回合内保持连续传播：

$$\mathbf{h}_t = \begin{cases} \mathbf{0} & \text{若} t = t_{\text{episode_start}} \\ \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}_t \mathbf{x}_t & \text{若} t > t_{\text{episode_start}} \end{cases} \tag{4.9}$$

图 4.3给出状态生命周期的状态机表示。

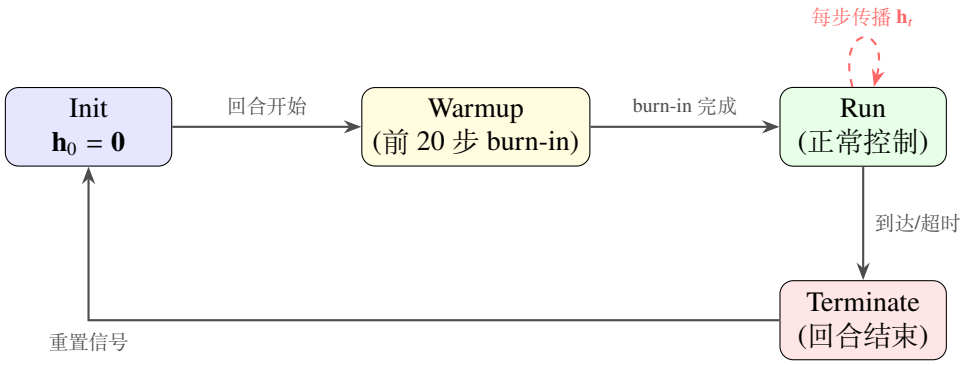

图 4.3　状态生命周期状态机：Init→Warmup→Run→Terminate→Reset

Batch/Streaming 时间轴对比示意见图 4.4。

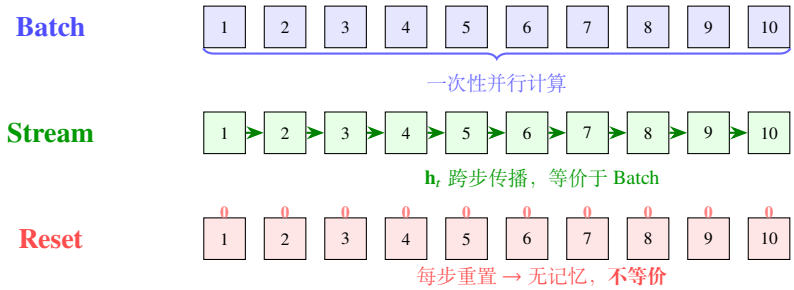

图 4.4　Batch 模式与正确/错误 Streaming 模式的时间轴对比

4.3.3　实现细节

关键实现细节包括：

- Mamba 卷积缓存：Mamba 模块内部的 1D 卷积层（$d_{\text{conv}} = 4$）维护一个长度为 $d_{\text{conv}} - 1 = 3$ 的输入缓存。该缓存同样需要在回合边界清零、回合内持续更新。遗漏卷积缓存的重置不会导致碰撞率飙升（因其影响仅持续 3 步），但会在回合起始引入约 3 步的输出偏差；
- 序列偏移（seqlen_offset）：Mamba 的某些位置编码实现依赖 seqlen_offset 指示当前处于序列中的绝对位置。若该计数器未正确累加或被意外重置，可能导致位置编码错误；
- Python 框架的陷阱：在 PyTorch 中，`model.eval()` 仅影响 Dropout 和 BatchNorm 的行为，不会自动处理序列模型的内部状态。状态管理是用户代码的责任。

算法 6给出状态生命周期管理的完整实现。

算法 6 回合边界级状态生命周期管理

输入: 策略网络 π，推理参数 `inf_params`

```
// 在仿真器 reset 信号触发时调用
procedure OnEpisodeReset
    inf_params.state ← 0                                  ▷ 清零内部状态
    inf_params.seqlen_offset ← 0                          ▷ 重置序列偏移
    inf_params.conv_state ← 0                             ▷ 清零 Mamba 卷积缓存
    reset_count ← reset_count + 1                         ▷ 记录重置次数
end procedure

// 在每个控制步调用
procedure OnControlStep(x_t)
    assert inf_params.seqlen_offset ≥ 0                   ▷ 硬防护：偏移合法
    y_t ← π.forward(x_t, inf_params)                      ▷ 前向推理
                                  ▷ 状态由 forward 内部自动更新至 inf_params
    inf_params.seqlen_offset ← inf_params.seqlen_offset + 1
    return y_t
end procedure
```

4.3.4 常见工程错误与症状对照

表 4.2梳理了实践中观察到的四类典型状态管理错误及其症状。

表 4.2 常见状态管理工程错误与症状对照

错误类型	根因	症状表现	诊断方法
每步状态重置	推理循环中在每次 `forward` 前显式调用 `h=zeros()`	碰撞率↑↑↑，Jerk↑，系统性漂移	等价性单测：$\Delta\mathbf{v}_t \gg 10^{-5}$
seqlen_offset 未累加	回合内 offset 固定为 0 或被意外重置	位置编码错误，输出周期性异常	检查 offset 是否单调递增
数值精度/确定性不一致	训练 float32、推理 float16，或未开启 CUDA 确定性模式	输出微小偏差逐步累积为宏观漂移	Batch-Stream $\Delta\mathbf{v}_t$ 随 t 线性增长
多线程竞争条件	状态更新与读取在不同线程中并发执行，无锁保护	偶发性输出跳变（难以复现）	单线程模式下 $\Delta\mathbf{v}_t < 10^{-5}$，多线程下偶发 $\Delta\mathbf{v}_t \gg 10^{-5}$

其中，每步状态重置是最严重的错误（直接导致碰撞率从 0%→90%），也是最容易在不经意间引入的——例如在推理循环中调用封装函数时，函数内部为保证” 无副

作用” 而创建了新的状态张量。

数值精度不一致是最隐蔽的错误：float16 推理在单步上的误差可能仅为 10^{-3} 量级，但通过递推累积 T 步后（$T = 150$），总误差可达 $O(T \cdot 10^{-3}) = O(10^{-1})$ 量级，足以导致控制行为偏差。该研究实验统一使用 float32 精度以消除此类风险。

4.4 等价性单测与硬防护机制

4.4.1 等价性单测

给定一条测试轨迹 $\{\mathbf{x}_1, \ldots, \mathbf{x}_T\}$，分别以 Batch 和 Streaming 模式前向计算，比较逐步输出差异：

$$\Delta \mathbf{v}_t = \|\mathbf{y}_t^{\text{batch}} - \mathbf{y}_t^{\text{stream}}\|_2 \tag{4.10}$$

正确实现下 $\Delta \mathbf{v}_t$ 应在浮点精度范围内（$< 10^{-5}$）。

算法 7给出等价性单测的伪代码。

算法 7 Batch–Streaming 等价性单测

输入: 策略网络 π，测试序列 $\{\mathbf{x}_1, \ldots, \mathbf{x}_T\}$，阈值 ϵ

$\{\mathbf{y}_1^{\text{batch}}, \ldots, \mathbf{y}_T^{\text{batch}}\} \leftarrow \pi.\text{batch_forward}(\{\mathbf{x}_1, \ldots, \mathbf{x}_T\}, \mathbf{0})$
$\mathbf{h} \leftarrow \mathbf{0}$
for $t = 1$ to T **do**
    $\mathbf{y}_t^{\text{stream}}, \mathbf{h} \leftarrow \pi.\text{stream_forward}(\mathbf{x}_t, \mathbf{h})$
end for
$\Delta_{\max} \leftarrow \max_{1 \le t \le T} \|\mathbf{y}_t^{\text{batch}} - \mathbf{y}_t^{\text{stream}}\|_2$
if $\Delta_{\max} < \epsilon$ **then**
    return PASS
else
    return FAIL ($\Delta_{\max}$，$t^* = \arg\max_t \|\mathbf{y}_t^{\text{batch}} - \mathbf{y}_t^{\text{stream}}\|_2$)
end if

4.4.2 阈值选择依据

等价性阈值 $\epsilon = 10^{-5}$ 的选取基于 float32 浮点运算的误差分析：

- float32 的机器精度（machine epsilon）为 $\epsilon_{\text{mach}} \approx 1.19 \times 10^{-7}$；
- 对于包含 $d_{\text{model}} = 192$ 维矩阵-向量乘法的单步递推，理论最大浮点累积误差约为 $O(\sqrt{d_{\text{model}}} \cdot \epsilon_{\text{mach}}) \approx O(10^{-6})$；

- 经 4 层 Mamba 的级联递推，单步误差上界约为 $4 \times O(10^{-6}) \approx O(10^{-5})$。

因此 $\epsilon = 10^{-5}$ 既能容纳正常的浮点误差累积，又能检测到任何状态管理级别的错误（该类错误通常导致 $\Delta_t > 10^{-1}$，与阈值差 5 个数量级以上）。

等价性测试配置与通过标准见表 4.3。

表 4.3　等价性测试配置与通过标准

参数	设置	说明
测试轨迹长度	150 步	与训练序列长度一致
测试轨迹数量	10 条	覆盖不同速度档
模型精度	float32	训练与推理精度对齐
CUDA 确定性	`torch.use_deterministic_algorithms(True)`	消除非确定性运算
等价性阈值	$\Delta \mathbf{v}_t < 10^{-5}$	基于浮点误差分析

4.4.3　硬防护机制

硬防护机制旨在将” 隐蔽的工程 Bug” 转化为” 可立即检测的运行时错误”，包含三个层级：

1. 运行时断言（Assertion）：每个控制步前检查推理参数的合法性——seqlen_offset 是否单调递增、状态张量形状是否匹配、当前是否处于已知的安全模式。断言失败触发 fail-fast 立即终止，避免产生错误数据；
2. 配置锁定（Config Lock）：评测开始时将关键配置（模型路径、权重哈希、推理精度、RACS 参数等）写入日志并锁定，运行中任何修改尝试触发告警；
3. 可审计日志（Audit Log）：记录完整的运行时信息，用于事后审计与问题定位。

表 4.4给出可审计日志的字段定义。

这套硬防护机制的设计理念是将信任建立在可验证的机制上，而非开发者的记忆力上。在协作开发或代码审查中，任何人都可以通过审计日志独立验证实验结果的状态管理正确性。

4.5　案例研究与实验

该节评测协议引用第 2 章表 2.5。所有实验使用完全相同的策略权重，唯一变量是状态管理方式或频率。

表 4.4　可审计日志字段定义

字段名	含义	示例值
model_weight_hash	模型权重文件的 SHA-256 哈希	a3f2...c7e1
inference_dtype	推理精度	float32
cuda_deterministic	CUDA 确定性模式	True
state_management	状态管理模式	KeepState / ResetState
episode_reset_times	回合重置时刻列表	[0, 4502, 9015, ...]
seqlen_offset_trace	序列偏移计数器轨迹	[0, 1, 2, ..., 4501, 0, 1, ...]
racs_params	RACS 超参数	{delta_max: 2.0, ...}
eval_seed	评测随机种子	42
env_config	环境配置摘要	{density: 0.5, ...}
eval_start_time	评测开始时间	2025-01-15T10:30:00Z

4.5.1　KeepState 与 ResetState 对比

为验证流式状态管理对 Mamba 策略稳定性的影响，本节设计状态管理消融实验。KeepState 为正确模式，仅在回合边界重置内部状态；ResetState 为错误模式，在每个控制步将内部状态重置为零向量。

表 4.5　Mamba 流式状态管理消融实验（KeepState vs ResetState，Spheres 7 m/s）

模式	Collision Rate (%)	Mean Jerk (m/s)	Mean Y Drift (m)
Mamba (KeepState)	0.0	0.198	0.022
Mamba (ResetState)	90.0	0.376	0.770

实验结果如表 4.5所示：ResetState 模式导致碰撞率由 0% 骤升至 90%，策略基本丧失避障能力；该退化并非输出随机化，而是产生缺乏时间连续性的控制序列，使得局部合理决策在全局尺度上无法形成稳定轨迹。

在控制平滑性方面：Mean Jerk 由 0.198 m/s 上升至 0.376 m/s，增幅约 90%；这一现象与理论分析一致——当内部状态被逐步清零后，模型退化为逐帧映射结构，无法对历史观测进行滤波与积分，深度噪声在控制指令中被放大，从而表现为明显抖动。

在轨迹稳定性方面：Mean Y Drift 由 0.022 m 增加至 0.770 m，约提升 34 倍；横向漂移的累积在密集障碍环境中会迅速放大风险，使无人机偏离安全通道并触发连锁碰撞。

整体结果表明：流式状态的持续维护是 Mamba 时序建模能力发挥的必要条件；逐步重置内部状态将直接破坏其时间一致性结构，导致系统级性能崩溃。

4.5.2 LSTM 的状态重置退化

为验证状态管理问题的跨架构普遍性，对 ViT+LSTM 基线进行相同的 KeepState/ResetState 消融。

结果见表 4.6。

表 4.6 LSTM 流式状态管理消融实验（KeepState vs ResetState，Spheres 7 m/s）

模式	Collision Rate (%)	Mean Jerk (m/s)	Mean Y Drift (m)
LSTM (KeepState)	6.93	0.42	0.064
LSTM (ResetState)	12.50	0.76	0.112

实验结果显示，LSTM 在 ResetState 下同样出现显著退化：Collision Rate 由 6.93% 上升至 12.50%，Mean Jerk 由 0.42 上升至 0.76，Mean Y Drift 由 0.064 增至 0.112。这实验证实了状态管理问题与序列模型的具体架构无关——这是一个通用的流式部署风险。

4.5.3 漂移可视化

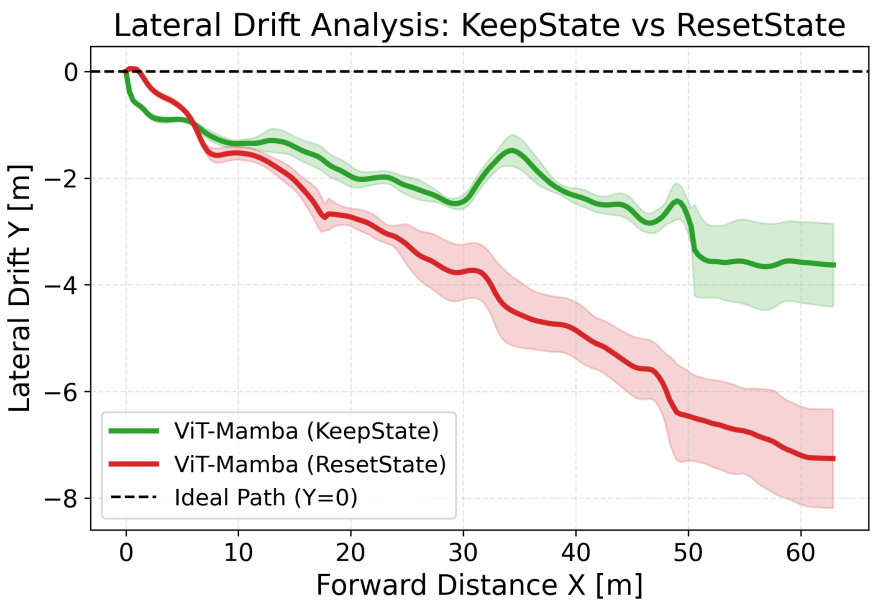

图 4.5 KeepState 与 ResetState 的漂移对比。ResetState（红色）导致显著的横向漂移趋势，而 KeepState（绿色）的轨迹保持稳定。

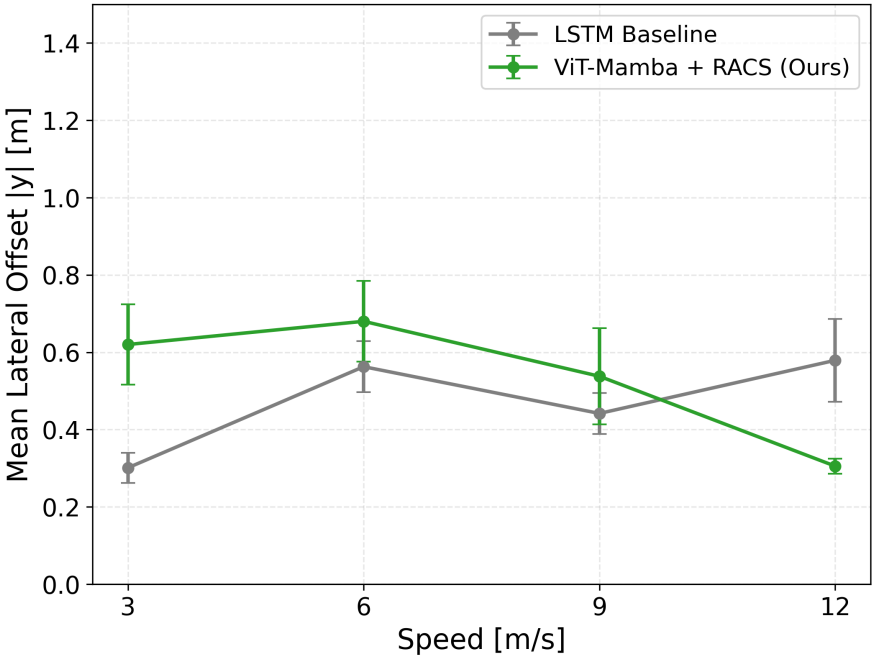

图 4.6　KeepState 与 ResetState 模式下横向漂移的累积对比

图 4.5和图 4.6直观展示了 ResetState 导致的系统性漂移。0.770 m 的平均横向偏移在密集障碍环境（障碍间距 ~ 2 m）中意味着无人机的有效安全通道宽度被” 吃掉” 了约 38%，碰撞概率的急剧上升因而不可避免。

4.5.4 等价性单测结果

在正确的 KeepState 实现下，Batch 与 Streaming 模式输出差异 $\Delta\mathbf{v}_t$ 在 10^{-6} 量级，远低于 $\epsilon = 10^{-5}$ 的阈值，确认两种模式的数学等价性未被工程实现破坏。图 4.7给出 $\Delta\mathbf{v}_t$ 随时间步的分布。

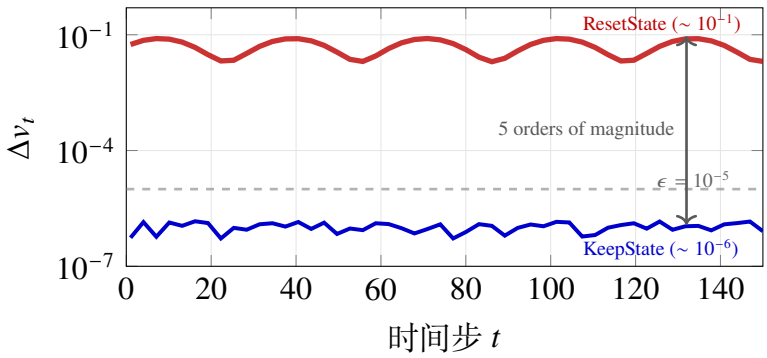

图 4.7 Batch–Streaming 等价性测试：KeepState 下 $\Delta\mathbf{v}_t$ 在 10^{-6} 量级（蓝色），ResetState 下 $\Delta\mathbf{v}_t$ 在 10^{-1} 量级（红色），两者差 5 个数量级

可以看到：KeepState 的 $\Delta\mathbf{v}_t$ 稳定在 10^{-6} 附近，远低于阈值（虚线）；而 ResetState 的 $\Delta\mathbf{v}_t$ 高达 10^{-1} 量级，超出阈值 4 个数量级——等价性单测可以在第一个时间步即检测到问题。

4.5.5 重置频率消融

为进一步理解状态重置的影响，该节考察”每 k 步重置一次”($k = 1, 5, 10, 20, 50, \infty$) 的退化曲线。$k = 1$ 对应 ResetState，$k = \infty$ 对应 KeepState。

消融结果汇总见图 4.8。

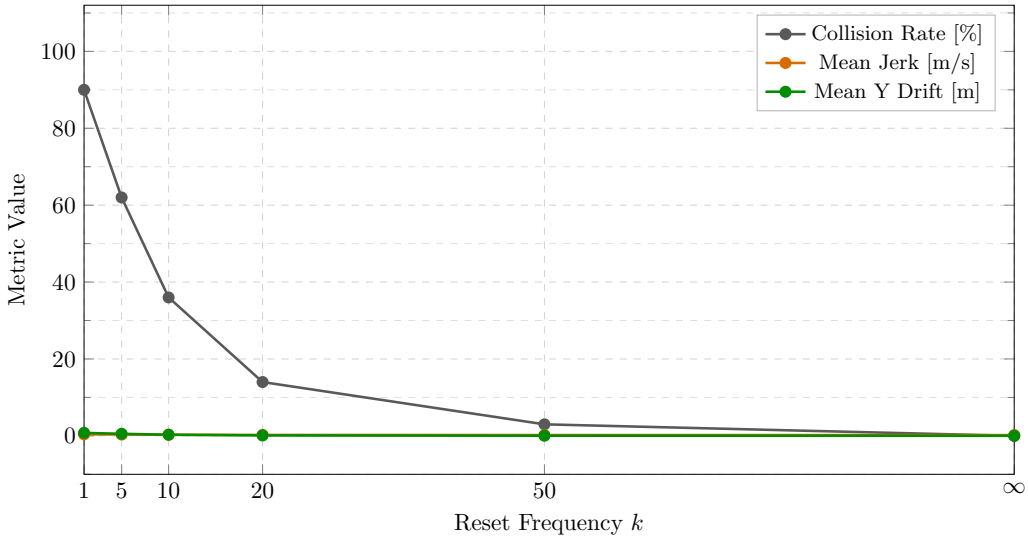

图 4.8　重置频率消融实验结果：重置越频繁，碰撞率、Jerk 与横向漂移越高。

实验结果显示：碰撞率随 k 增大单调递减且呈明显非线性，在 $k = 1, 5, 10, 20, 50, \infty$ 下分别为 90.0、62.0、36.0、14.0、3.0、0.0。同样，Mean Jerk 由 0.376 逐步降至 0.198，Mean Y Drift 由 0.770 逐步降至 0.022。该结果表明系统存在任务相关的有效记忆窗口：当重置频率降低到一定程度后，性能快速逼近 KeepState 上界。

4.5.6　部署 burn-in 消融

第 3 章的训练 burn-in（前 20 步不计入损失）是训练侧的设计。该节考察部署侧的 burn-in 效果：在回合开始后的前 b 步内，虽然模型正常推理并更新状态，但控制指令由专家策略提供（或固定为匀速前进），以等待隐状态” 热身” 到稳定值。

消融结果汇总见图 4.9。实验结果表明，b 从 0 增加到 20 时，Collision Rate 由 3.36 降至 2.49，Mean Jerk 由 1.14 降至 0.98，Mean Y Drift 由 0.089 降至 0.066；部署 burn-in 对安全性有小幅改善，同时对平滑性和漂移控制有稳定收益。

部署 burn-in 的作用机理是：Mamba 隐状态 $\mathbf{h}_t$ 在零初始化后需要若干步输入完成稳定化。本实验已验证，在该稳定化阶段加入热身策略可降低早期不稳定输出带来的累积偏差。对于该研究仿真环境（起始处通常为开阔区域），其收益体现在平滑性与漂移指标上；在复杂初始位姿场景中，这一机制可进一步提升部署鲁棒性。

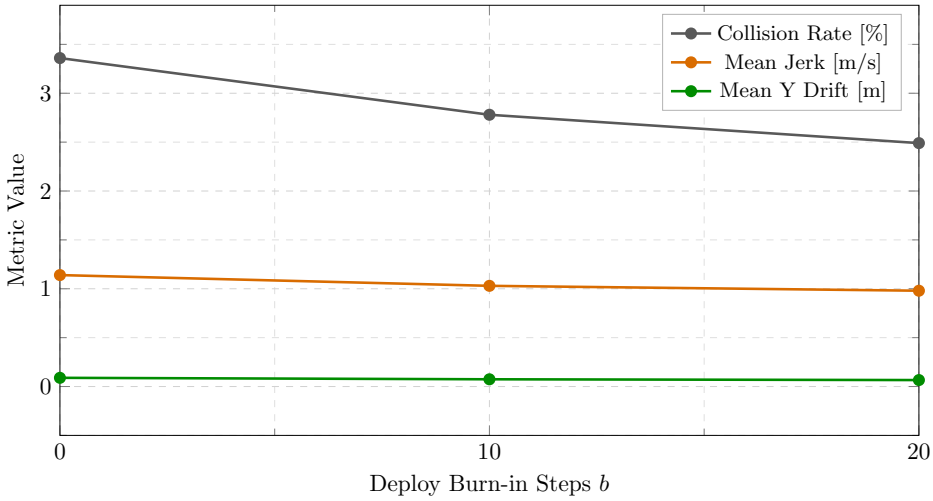

图 4.9　部署 burn-in 消融实验结果：适度 burn-in 对碰撞率改善有限，但可稳定降低 Jerk 与漂移。

4.5.7　跨速度泛化验证

为验证状态管理问题在不同速度条件下的一致性，图 4.10和图 4.11 分别给出 KeepState 与 ResetState 在多个速度档位下的碰撞率与横向漂移对比。

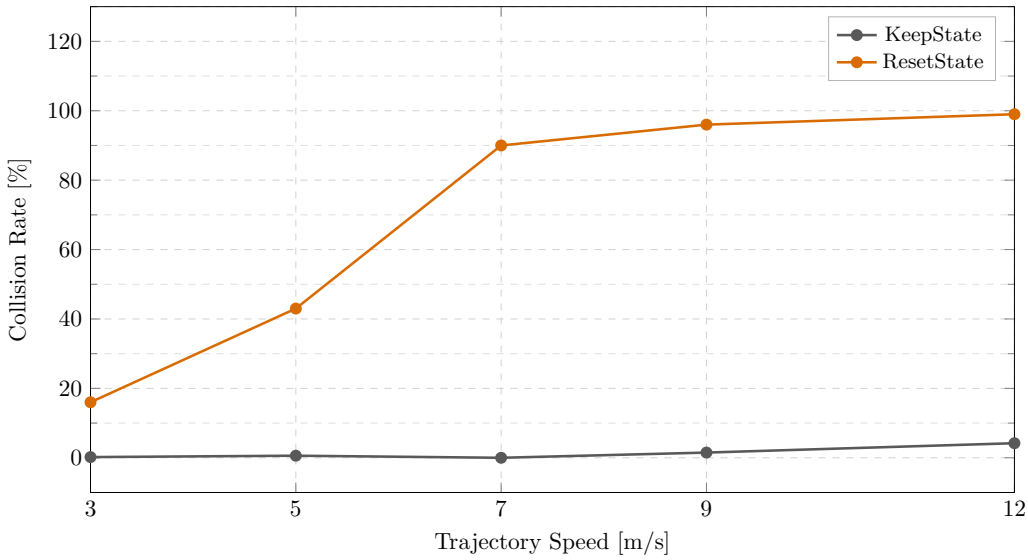

图 4.10　KeepState 与 ResetState 跨速度碰撞率对比：ResetState 的碰撞率随速度上升快速放大。

从图 4.10可见，ResetState 在全速度范围内的 Collision Rate 均显著劣于 KeepState，且退化随速度上升快速放大。在 3 m/s 时 Collision Rate 为 16.0%（KeepState 为 0.2%），到 12 m/s 时升至 99.0%（KeepState 为 4.2%）；从图 4.11可见，ResetState 的 Mean Y Drift 由 0.182 增至 1.480，而 KeepState 仅由 0.010 增至 0.055。这说明低速测试会低估

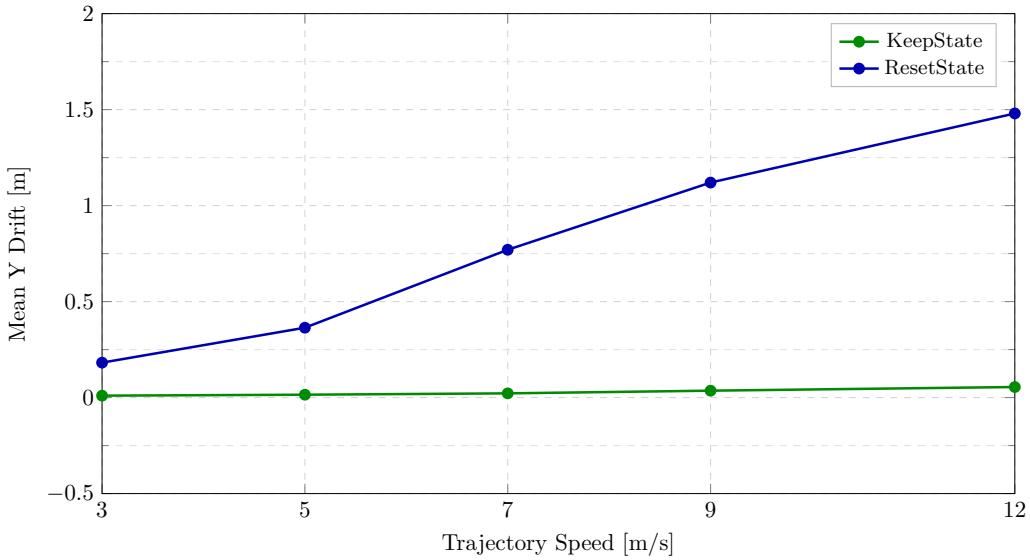

图 4.11　KeepState 与 ResetState 跨速度横向漂移对比：ResetState 在全速度域均表现出更大的横向漂移。

状态管理错误的风险，而高速工况能充分暴露其灾难性后果。

4.6　第 4 章小结

第 4 章系统分析了序列模型在流式部署中的状态一致性问题，揭示了一个对所有使用序列模型进行端到端控制的研究具有普遍警示意义的关键陷阱：

1. 理论贡献：给出了 Batch–Streaming 等价性的形式化定义、充要条件与归纳证明，分析了 SSM、LSTM、Transformer 三类架构的状态管理类比，建立了跨架构的通用理论框架；

2. 实验证据：碰撞率从 0% 飙升至 90%、Jerk 增加 90%、Y-Drift 增加 34 倍的实验数据，以无可争辩的方式证明了状态管理错误的毁灭性后果。重置频率消融揭示了” 临界记忆长度” 的存在；

3. 工程方法论：回合边界级状态生命周期管理协议、常见错误症状对照表、等价性单测与硬防护机制（断言 + 配置锁定 + 可审计日志），将” 隐蔽的工程 Bug” 转化为” 可检测的运行时错误”。

部署一致性由经验问题转化为可验证问题。这一方法论对所有依赖内部状态递推的序列模型均具有普适价值，尤其是在安全关键的实时控制应用中。

在确保部署一致性的基础上，第 5 章将进一步探索更高效的视觉骨干：将空间编码器从 ViT 替换为 MambaVision，考察全 SSM 架构（空间 SSM + 时序 SSM）的可行性与能力边界。

参考文献

[1] 刘小雄, 李文皓, 申功璋. 四旋翼无人机自主导航与避障技术综述[J]. 航空学报, 2020, 41(10): 1–22.

[2] Mur-Artal R, Tardós J D. Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras[J]. IEEE Transactions on Robotics, 2017, 33(5): 1255–1262.

[3] Qin T, Li P, Shen S. Vins-mono: A robust and versatile monocular visual-inertial state estimator[J]. IEEE Transactions on Robotics, 2018, 34(4): 1004–1020.

[4] Campos C, Elvira R, Rodriguez J J G, et al. ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM[J]. IEEE Transactions on Robotics, 2021, 37(6): 1874–1890.

[5] Karaman S, Frazzoli E. Sampling-based algorithms for optimal motion planning[C]. The International Journal of Robotics Research. [S.l.: s.n.], 2011.

[6] Zhou B, Gao F, Wang L, et al. Robust and efficient quadrotor trajectory generation for fast autonomous flight[J]. IEEE Robotics and Automation Letters, 2019, 4(4): 3529–3536.

[7] Loquercio A, Kaufmann E, Ranftl R, et al. Learning high-speed flight in the wild[J]. Science Robotics, 2021, 6(59).

[8] Pomerleau D A. ALVINN: An autonomous land vehicle in a neural network[J]. Advances in Neural Information Processing Systems (NeurIPS), 1989.

[9] Bojarski M, Del Testa D, Dworakowski D, et al. End to end learning for self-driving cars[C]. arXiv preprint arXiv:1604.07316. [S.l.: s.n.], 2016.

[10] Loquercio A, Maqueda A I, Del Blanco C R, et al. Dronet: Learning to fly by driving[C]. IEEE Robotics and Automation Letters: volume 3. 2018: 1088–1095.

[11] Sadeghi F, Levine S. CAD2RL: Real single-image flight without a single real image[C]. Robotics: Science and Systems (RSS). [S.l.: s.n.], 2017.

[12] Kaufmann E, Loquercio A, Ranftl R, et al. Deep drone racing: Learning agile flight in dynamic environments[C]. Proceedings of the Conference on Robot Learning (CoRL). [S.l.: s.n.], 2018.

[13] Kaufmann E, Bauersfeld L, Loquercio A, et al. Champion-level drone racing using deep reinforcement learning[J]. Nature, 2023.

[14] Levine S, Finn C, Darrell T, et al. End-to-end training of deep visuomotor policies[J]. Journal of Machine Learning Research, 2016, 17(39): 1–40.

[15] Chen D, Zhou B, Koltun V, et al. Learning by cheating[C]. Proceedings of the Conference on Robot

Learning (CoRL). [S.l.: s.n.], 2020.

[16] Bhattacharya A, Rao N, Parikh D, et al. Vision transformers for end-to-end vision-based quadrotor obstacle avoidance[C]. 2025 IEEE International Conference on Robotics and Automation (ICRA). [S.l.]: IEEE, 2025: 1596–1603.

[17] Gu A, Dao T. Mamba: Linear-time sequence modeling with selective state spaces[J]. arXiv preprint arXiv:2312.00752, 2023.

[18] Elman J L. Finding structure in time[J]. Cognitive Science, 1990, 14(2): 179–211.

[19] Hochreiter S, Schmidhuber J. Long short-term memory[J]. Neural Computation, 1997, 9(8): 1735–1780.

[20] Ross S, Gordon G, Bagnell D. A reduction of imitation learning and structured prediction to no-regret online learning[C]. Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS). [S.l.: s.n.], 2011.

[21] Dosovitskiy A, Beyer L, Kolesnikov A, et al. An image is worth 16x16 words: Transformers for image recognition at scale[J]. arXiv preprint arXiv:2010.11929, 2020.

[22] Zhu L, Liao B, Zhang Q, et al. Vision mamba: Efficient visual representation learning with bidirectional state space model[C]. International Conference on Machine Learning (ICML). [S.l.: s.n.], 2024.

[23] Liu Y, Tian Y, Zhao Y, et al. Vmamba: Visual state space model[J]. arXiv preprint arXiv:2401.10166, 2024.

[24] Hatamizadeh A, Kautz J. Mambavision: A hybrid mamba-transformer vision backbone[C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). [S.l.: s.n.], 2025.

[25] Mahony R, Kumar V, Corke P. Multirotor aerial vehicles: Modeling, estimation, and control of quadrotor[J]. IEEE Robotics & Automation Magazine, 2012, 19(3): 20–32.

[26] Song Y, Sun J, Liu S, et al. Flightmare: A flexible quadrotor simulator[C]. Proceedings of the Conference on Robot Learning (CoRL). [S.l.: s.n.], 2021.

[27] Lee T, Leok M, McClamroch N H. Geometric tracking control of a quadrotor UAV on SE(3)[J]. IEEE Conference on Decision and Control (CDC), 2010: 5420–5425.

[28] Codevilla F, Müller M, López A, et al. End-to-end driving via conditional imitation learning[J]. IEEE International Conference on Robotics and Automation (ICRA), 2018.

[29] Osa T, Paull J, Devlin J, et al. An algorithmic perspective on imitation learning[J]. Foundations and Trends in Robotics, 2018, 7(1–2): 1–179.

[30] Codevilla F, Santana E, López A M, et al. Exploring the limitations of behavior cloning for autonomous driving[C]. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). [S.l.: s.n.], 2019.

[31] Zhang J, Cho K. Query-efficient imitation learning for end-to-end simulated driving[C]. Proceedings of the AAAI Conference on Artificial Intelligence. [S.l.: s.n.], 2017.

[32] Kelly M, Sidrane C, Driggs-Campbell K, et al. HG-DAgger: Interactive imitation learning with human experts[C]. IEEE International Conference on Robotics and Automation (ICRA). [S.l.: s.n.], 2019.

[33] LeCun Y, Bottou L, Bengio Y, et al. Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998, 86(11): 2278–2324.

[34] Simonyan K, Zisserman A. Very deep convolutional networks for large-scale image recognition[J]. International Conference on Learning Representations (ICLR), 2015.

[35] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition[J]. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016: 770–778.

[36] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[J]. Advances in Neural Information Processing Systems (NeurIPS), 2017.

[37] Xing Z, Romero A, Scaramuzza D. Vision transformers for end-to-end vision-based quadrotor obstacle avoidance[J]. arXiv preprint arXiv:2405.10391, 2024.

[38] Touvron H, Cord M, Douze M, et al. Training data-efficient image transformers & distillation through attention[C]. International Conference on Machine Learning (ICML). [S.l.: s.n.], 2021.

[39] Liu Z, Lin Y, Cao Y, et al. Swin transformer: Hierarchical vision transformer using shifted windows [C]. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). [S.l.: s.n.], 2021: 10012–10022.

[40] He K, Chen X, Xie S, et al. Masked autoencoders are scalable vision learners[C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). [S.l.: s.n.], 2022: 16000–16009.

[41] Bao H, Dong L, Piao S, et al. BEiT: BERT pre-training of image transformers[C]. International Conference on Learning Representations (ICLR). [S.l.: s.n.], 2022.

[42] Gu A, Goel K, Re C. Efficiently modeling long sequences with structured state spaces[J]. International Conference on Learning Representations (ICLR), 2022.

[43] Gu A, Dao T, Ermon S, et al. Hippo: Recurrent memory with optimal polynomial projections[C]. Advances in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2020.

[44] Smith J T, Warrington A, Linderman S W. Simplified state space layers for sequence modeling[C]. International Conference on Learning Representations (ICLR). [S.l.: s.n.], 2023.

[45] Gu A, Gupta A, Goel K, et al. On the parameterization and initialization of diagonal state space models[J]. Advances in Neural Information Processing Systems (NeurIPS), 2022.

[46] Dao T, Gu A. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality[J]. International Conference on Machine Learning (ICML), 2024.

[47] Shah S, Dey D, Lovett C, et al. AirSim: High-fidelity visual and physical simulation for autonomous vehicles[C]. Field and Service Robotics (FSR). [S.l.: s.n.], 2018.

[48] Furrer F, Burri M, Achtelik M, et al. RotorS—a modular gazebo MAV simulator framework[J]. Robot Operating System (ROS), 2016: 595–625.

[49] Foehn P, Kaufmann E, Romero A, et al. Agilicious: Open-source and open-hardware agile quadrotor for vision-based flight[J]. Science Robotics, 2022, 7(67).

[50] Gandhi D, Pinto L, Gupta A. Learning to fly by crashing[J]. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2017.

[51] Kaufmann E, Loquercio A, Ranftl R, et al. Deep drone acrobatics[C]. Robotics: Science and Systems (RSS). [S.l.: s.n.], 2020.

[52] Pan Y, Cheng C A, Saigol K, et al. Agile autonomous driving using end-to-end deep imitation learning [C]. Robotics: Science and Systems (RSS). [S.l.: s.n.], 2018.

[53] Shah D, Sridhar A, Bhorkar A, et al. GNM: A general navigation model to drive any robot[J]. IEEE International Conference on Robotics and Automation (ICRA), 2023.

[54] Engel J, Schöps T, Cremers D. LSD-SLAM: Large-scale direct monocular SLAM[C]. European Conference on Computer Vision (ECCV). [S.l.: s.n.], 2014.

[55] Cadena C, Carlone L, Carrillo H, et al. Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age[J]. IEEE Transactions on Robotics, 2016, 32(6): 1309–1332.

[56] Mellinger D, Kumar V. Minimum snap trajectory generation and control for quadrotors[J]. IEEE International Conference on Robotics and Automation (ICRA), 2011: 2520–2525.

[57] Richter C, Bry A, Roy N. Polynomial trajectory planning for aggressive quadrotor flight in dense indoor environments[J]. International Symposium of Robotics Research (ISRR), 2016.

[58] Kamel M, Stastny T, Alexis K, et al. Model predictive control for trajectory tracking of unmanned aerial vehicles using ROS[J]. Robot Operating System (ROS), 2017: 3–39.

[59] Neunert M, de Crousaz C, Furrer F, et al. Fast nonlinear model predictive control for unified trajectory

optimization and tracking[J]. IEEE International Conference on Robotics and Automation (ICRA), 2016.

[60] 高翔, 张涛, 刘毅, 等. 视觉 SLAM 研究进展[J]. 机器人, 2019, 41(6): 768–776.

[61] 张弓, 方正, 陈思哲. 移动机器人视觉惯性 SLAM 研究进展[J]. 自动化学报, 2018, 44(5): 769–786.

[62] 吴潇, 刘勇, 赵鹏. 面向无人机的视觉 SLAM 方法综述[J]. 计算机工程与应用, 2022, 58(12): 1–15.

[63] Zhou X, Wang Z, Ye H, et al. EGO-Planner: An ESDF-free gradient-based local planning framework for quadrotors[J]. IEEE Robotics and Automation Letters, 2021, 6(2): 478–485.

[64] 何承坤, 戴明, 张瑞. 四旋翼无人机轨迹优化与跟踪控制研究综述[J]. 控制与决策, 2021, 36(7): 1537–1556.

[65] 张涛, 邢强, 宋博. 无人机自主飞行控制技术研究进展与展望[J]. 自动化学报, 2020, 46(2): 231–248.

[66] Brunke L, Greeff M, Hall A, et al. Safe learning in robotics: From learning-based control to safe reinforcement learning[J]. Annual Review of Control, Robotics, and Autonomous Systems, 2022, 5: 411–444.

[67] Ames A D, Coogan S, Egerstedt M, et al. Control barrier functions: Theory and applications[J]. European Control Conference (ECC), 2019: 3420–3431.

[68] Cheng R, Orosz G, Murray R M, et al. End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks[J]. Proceedings of the AAAI Conference on Artificial Intelligence, 2019.

[69] Wabersich K P, Zeilinger M N. Linear model predictive safety certification for learning-based control [J]. IEEE Conference on Decision and Control (CDC), 2018.

[70] Fisac J F, Akametalu A K, Zeilinger M N, et al. A general safety framework for learning-based control in uncertain robotic systems[J]. IEEE Transactions on Automatic Control, 2019, 64(7): 2737–2752.

[71] García J, Fernández F. A comprehensive survey on safe reinforcement learning[J]. Journal of Machine Learning Research, 2015, 16(1): 1437–1480.

[72] 雷志勇, 陈世超, 刘尚旺. 基于深度强化学习的无人机视觉避障方法[J]. 自动化学报, 2020, 46 (4): 796–805.

[73] 严旭, 吴庆宪, 姜斌. 基于深度强化学习的无人机避障算法研究[J]. 控制理论与应用, 2021, 38 (10): 1697–1706.

[74] 李超, 张晓明, 王春. 基于强化学习的无人机路径规划方法综述[J]. 系统工程与电子技术, 2022,

44(9): 2772–2783.

[75] 陈杰, 李擎, 王莉. 深度强化学习在无人机自主导航中的研究进展[J]. 航空学报, 2023, 44(4): 1–18.

[76] 朱福利, 杨华, 刘子明. 深度学习在无人机视觉感知中的应用综述[J]. 计算机学报, 2021, 44(6): 1231–1255.

[77] Tobin J, Fong R, Ray A, et al. Domain randomization for transferring deep neural networks from simulation to the real world[J]. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2017.

[78] Peng X B, Andrychowicz M, Zaremba W, et al. Sim-to-real transfer of robotic control with dynamics randomization[J]. IEEE International Conference on Robotics and Automation (ICRA), 2018.

[79] Molchanov A, Chen T, Honig W, et al. Sim-to-(multi)-real: Transfer of low-level robust control policies to multiple quadrotors[J]. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2019.

[80] Zhao W, Queralta J P, Westerlund T. Sim-to-real transfer in deep reinforcement learning for robotics: a survey[J]. IEEE Symposium Series on Computational Intelligence (SSCI), 2020.

[81] Loshchilov I, Hutter F. Decoupled weight decay regularization[C]. International Conference on Learning Representations (ICLR). [S.l.: s.n.], 2019.

[82] Blelloch G E. Prefix sums and their applications[J]. Synthesis of Parallel Algorithms, 1990: 35–60.